

# Polymarket Signal Analysis: Empirical Detection of Unusual Market Stress Events and Sniper Attacks

**Final Report - CS6290 Course Project**

**Date:** April 2026

---

## Table of Contents

1. Introduction
  - 1.1 Problem Statement & Scope
  - 1.2 Repository Link
2. Specification-Driven Development & System Design
  - 2.1 System Design Overview
3. Threat Model & Security Analysis
  - 3.1 Threat Model Definition
  - 3.2 Threat to Test/M Measurement Mapping
4. Core Implementation: Anomaly Detection Layers
  - 4.1 Infrastructure & Data Pipeline
  - 4.2 Type A - Market Stress Symptom Detection
  - 4.3 Type B - Sniper Detection Clue Extraction
  - 4.4 Visualization & Interactive Evidence View
5. Validation, Testing & Empirical Analysis
  - 5.1 Dataset & Empirical Approach
  - 5.2 Case Study Validation
6. Real-World Value & Conclusion
  - 6.1 Practical Value
  - 6.2 Conclusion

## 7. Appendix: AI Collaboration Statement

- 7.1 AI Toolchain and Responsibility Split
- 7.2 Skill-Based Workflow Applied
- 7.3 Constraint Control with AGENTS.md
- 7.4 Human Oversight and Decision Rights
- 7.5 Correction Case and Outcome

---

## 1. Introduction

### 1.1 Problem Statement & Scope

Prediction markets aggregate beliefs through trading, but the same trading record can also contain pressure signals that deserve investigation. Polymarket is permissionless and blockchain-based, which makes post-hoc review possible while making suspicious behavior harder to separate from ordinary volatility.

This project builds an offline investigative tool for historical Polymarket data. It does not predict prices, issue trading recommendations, execute orders, or monitor markets in real time. Its purpose is narrower: convert static CSV/JSON market records into interpretable anomaly evidence.

The system uses two complementary views. Type A identifies **Market Stress Anomalies** from 60-second Time Buckets with a 60-bucket Rolling Z-Score window and a 98th-percentile threshold. Type B identifies **Sniper Attack** patterns from wallet sessions with large buy-then-sell round trips and short holding periods. The detector covers six markets and is used for evidence presentation rather than automated action. Detailed mechanics appear in Chapters 3 and 4.

### 1.2 Repository Link

**Repository Link:** <https://github.com/zrk494/ProjectOfPrivacy-enhancingTech>  
**APP URL:** <https://polymarket-signal-analysis.streamlit.app/>

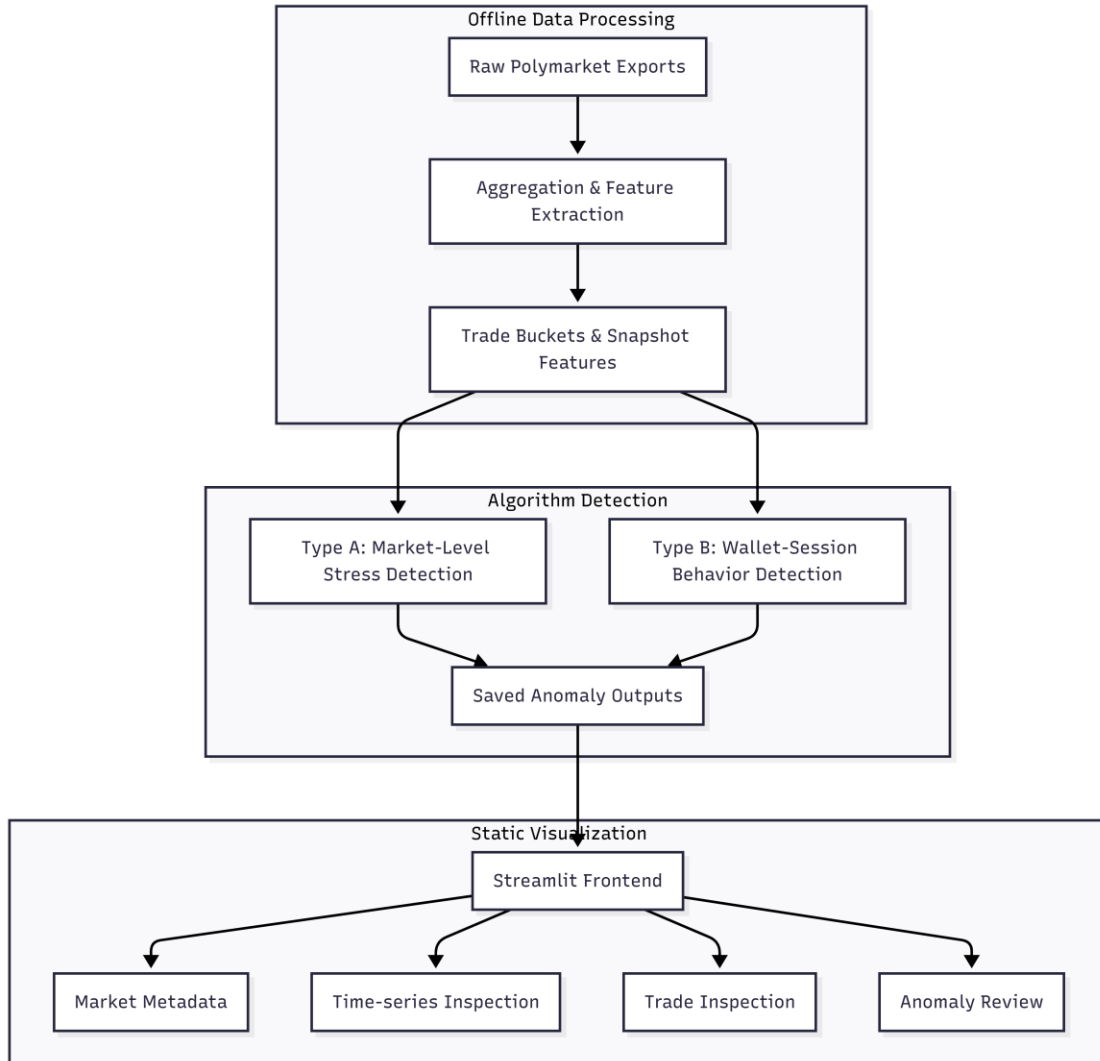
The repository contains the offline analysis pipeline, the historical datasets, and the Streamlit evidence interface used to inspect outputs.

---

## 2. Specification-Driven Development & System Design

### 2.1 System Design Overview

#### 2.1.1 System Architecture



The system is an offline pipeline with a static visualization layer. Raw Polymarket exports are aggregated into trade buckets and snapshot-derived features. Two independent detectors then run on the derived artifacts: Type A scores market-level stress, while Type B scores wallet-session behavior. A Streamlit frontend reads the saved outputs and provides market metadata, time-series inspection, trade inspection, and anomaly review. No database, persistent backend, or live data feed is required at runtime.

#### 2.1.2 Core Data Models

Three flat-file data models support the workflow.

| Data Model              | Key Attributes                                                                                                | Source File Pattern                     | Consumer                                                                     |
|-------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------|
| <b>Market Metadata</b>  | market_id,<br>market_name,<br>start_date,<br>end_date, outcome,<br>volume_usd                                 | market_metadata.csv (single file)       | Sidebar market selector; metadata view panel                                 |
| <b>Time Series Data</b> | datetime,<br>midpoint, spread,<br>bid_depth_top5,<br>ask_depth_top5,<br>depth_total,<br>imbalance             | {market_id}_{token_type}.csv per market | Time-series chart view; microstructure feature extraction                    |
| <b>Trade Data</b>       | timestamp, price,<br>size, amount_usd,<br>side,<br>trader_address (wallet),<br>transaction_hash,<br>market_id | {market_id}_{token_type}.csv per market | Trade table view; bucket aggregation; session splitting for sniper detection |

market\_id is the top-level join key. Time Series Data supports Market Stress Anomaly features. Trade Data supports both bucket aggregation and Session Splitting for Sniper Attack detection.

### 2.1.3 Key Design Decisions

- **Offline deployment.** The system reads pre-computed artifacts from disk. This removes runtime API dependence and matches a fixed six-market study.
- **60-second Time Buckets.** The bucket size keeps enough temporal detail for investigation while limiting sparse-bucket noise.
- **Dual-layer separation.** Type A measures market-wide symptoms. Type B measures wallet-level behavior. The split keeps interpretation clear.
- **Additive stress score.** The default Type A score sums normalized activity, liquidity, and price signals because the output is easier to interpret than a latent-factor score.
- **On-demand frontend loading.** The Streamlit layer loads only the files required for the selected market and view.
- **Session-based sniper analysis.** Wallet trades are split by a 30-minute inactivity gap so the detector evaluates behavioral episodes rather than isolated transactions.

---

## 3. Threat Model & Security Analysis

### 3.1 Threat Model Definition

#### 3.1.1 Two Attack Vectors

Two threat vectors define the analysis target.

**Market-wide stress manipulation** compresses abnormal trading activity into a short interval and distorts the visible market state. The measurable signature is simultaneous pressure in activity, liquidity, and price features. Type A addresses this vector through seven normalized signals and a Composite Stress Score.

**Sniper Attack** appears as a wallet-level round trip with large notional size, very short holding time, and near break-even exit. The measurable signature is a buy-then-sell session that is rare relative to the broader session population. Type B addresses this vector through Session Splitting, behavioral feature extraction, and Isolation Forest scoring.

#### 3.1.2 Real Case Reference

The Ethereum \$4,400 February target market (1303357\_YES) provides a concrete sniper example. Across six markets, the pipeline identified **40 strict sniper candidates** (has\_large\_trade=True, first\_side=BUY, last\_side=SELL, num\_trades>=2), and **3** passed secondary validation with is\_sniper=1 and a two-trade BUY-SELL structure. Rank 1 in the candidate file has 12 trades, so it is excluded from the canonical two-trade examples. The confirmed cases are summarized below.

| Rank | Wallet<br>(truncated)   | Holding<br>Period | Buy Amount  | Sell<br>Amount | Net P&L         | Anomaly Score  |
|------|-------------------------|-------------------|-------------|----------------|-----------------|----------------|
| 2    | 0x28d47763.<br>..ba55_1 | 74 s              | \$17,894.13 | \$17,876.22    | <b>-\$17.91</b> | <b>-0.4933</b> |
| 3    | 0x63b81ddc.<br>..f906_1 | 76 s              | \$17,754.11 | \$17,736.34    | <b>-\$17.77</b> | <b>-0.4870</b> |
| 4    | 0x6a4aaf27.<br>..f069_1 | 72 s              | \$17,787.85 | \$17,770.04    | <b>-\$17.81</b> | <b>-0.4847</b> |

These sessions are tightly clustered in duration, capital size, and small negative P&L. That pattern supports the sniper threat model and justifies a wallet-level detector.

### 3.2 Threat to Test/Masurement Mapping

The threat model is operationalized through direct measurement rules.

3.2.1 Threat-to-Detection Mapping Table

| Threat Vector       | Detection Rule                                                             | Key Metric(s)          |
|---------------------|----------------------------------------------------------------------------|------------------------|
| Volume manipulation | Rolling Z-Score (trade_count, window=60)                                   | z_trade_count          |
| Amount manipulation | Rolling Z-Score (amount_sum, window=60)                                    | z_amount_sum           |
| Large order anomaly | Rolling Z-Score (avg_trade_size, window=60)                                | z_avg_trade_size       |
| Liquidity shock     | Rolling Z-Score (spread + depth_drop, window=60)                           | z_spread, z_depth_drop |
| Price distortion    | Rolling Z-Score (abs_midpoint_return, window=60)                           | z_abs_midpoint_return  |
| Depth imbalance     | Rolling Z-Score (abs_depth_imbalance, window=60)                           | z_abs_depth_imbalance  |
| Sniper pattern      | Session Analysis + Isolation Forest (contamination=0.05, n_estimators=100) | anomaly_score          |

3.2.2 Type A Detection Chain: Market Stress Anomaly

Type A follows a fixed sequence: raw trades and snapshots are aggregated into 60-second Time Buckets, enriched with microstructure features, normalized with a 60-bucket Rolling Z-Score window, fused into activity/liquidity/price components, and thresholded at the 98th percentile. The output is a bucket-level Market Stress Anomaly flag for manual review.

3.2.3 Type B Detection Chain: Sniper Attack Detection

Type B groups trades by trader\_address, applies Session Splitting with a 30-minute inactivity gap, extracts behavioral features, embeds sessions, scores them with Isolation Forest, and then applies strict filters for buy-then-sell structure, large trade presence, and anomaly severity. The output is a ranked list of wallet-session candidates for forensic inspection.

---

## 4. Core Implementation: Anomaly Detection Layers

### 4.1 Infrastructure & Data Pipeline

The detection system consumes raw Polymarket market data through a three-stage pipeline: acquisition, preprocessing, and feature engineering. This section describes each stage.

#### 4.1.1 Data Acquisition and Storage

Data collection is implemented in the `pet-getdata` submodule under `submodules/data/pet-getdata`. The collector uses three endpoints with fixed roles:

- Gamma API (<https://gamma-api.polymarket.com>) for market discovery.
- CLOB API (<https://clob.polymarket.com>) for orderbook snapshots and primary trade retrieval.
- Data API (<https://data-api.polymarket.com>) as a fallback trade source.

The market selector starts from an initial pool of up to 100 markets and applies fixed filters: active status, binary YES/NO structure, and minimum liquidity (\$100). It then applies stratified sampling with a 3/2/1 split across high-, mid-, and low-liquidity tiers. The resulting study set contains six market IDs: 1303357, 1308327, 1345814, 1345822, 572469, and 572473.

Snapshot collection runs every 180 seconds. Each sample stores midpoint, best bid, best ask, spread, and top-5 bid/ask depth in per-market files under `polymarket_data/timeseries/`. Collection control parameters are fixed in configuration: market-to-market delay of 0.3 seconds, retry count of 3, and retry delay of 5 seconds.

Trade collection writes both YES and NO streams to `polymarket_data/trades/`. The collector uses paginated retrieval with page size 200 and a per-cycle safety cap of 20 pages per market. Cursor state is persisted in `trade_cursors.json` using timestamp-based cursors (`ts:{latest_timestamp}`), and write-time deduplication is enforced with `trade_id`.

The storage layout is flat-file only:

- `market_metadata.csv` for market-level attributes.
- `timeseries/{market_id}_YES.csv` for snapshot data.
- `trades/{market_id}_{YES|NO}.csv` for tick-level trades.
- `trade_cursors.json` for incremental fetch state.

#### 4.1.2 Preprocessing and Feature Construction

Raw trades and snapshots are normalized before anomaly scoring.

Before any analytical computation, both data streams pass through a deterministic preprocessing sequence:

**Step 1 -- Missing value handling.** Rows with null values in mandatory columns are removed. For trades, required fields include `timestamp`, `price`, and `size`. For

snapshots, required fields include timestamp, midpoint, spread, bid\_depth\_top5, and ask\_depth\_top5. If trade notional is missing, it is reconstructed as price × size.

**Step 2 -- Time alignment to UTC.** Timestamp fields are coerced to Unix epoch seconds in UTC.

**Step 3 -- Resampling into fixed-width time buckets.** Each record is mapped to a 60-second Time Bucket:

$$b = \left\lfloor \frac{t}{\Delta} \right\rfloor \cdot \Delta, \quad \Delta = 60 \text{ s}$$

Trade records are aggregated per bucket into trade\_count, amount\_sum, and avg\_trade\_size. Snapshot records are aligned by last-observation-carried-forward (LOCF), because snapshots arrive every 180 seconds while trades can be dense.

The fused table keeps seven core dimensions used by Type A:

| Feature             | Source    | Operational meaning             |
|---------------------|-----------|---------------------------------|
| trade_count         | trades    | activity frequency              |
| amount_sum          | trades    | total notional flow             |
| avg_trade_size      | trades    | ticket-size intensity           |
| spread              | snapshots | instantaneous transaction cost  |
| depth_drop          | snapshots | one-sided liquidity withdrawal  |
| abs_midpoint_return | snapshots | short-horizon price dislocation |
| abs_depth_imbalance | snapshots | bid/ask depth asymmetry         |

This output is a bucket-aligned offline dataset that feeds Type A market-level scoring (Section 4.2) and supports Type B session-level attribution (Section 4.3).

---

## 4.2 Type A - Market Stress Symptom Detection (Core Algorithm 1)

Type A detects market-level stress at the 60-second Time Bucket level. The unit of analysis is one bucket after trade and snapshot alignment. The output is a scalar stress\_score and a binary is\_stress\_anomaly flag per bucket.

The detector reports statistical symptoms. It does not assign intent or identity. Attribution is handled by Type B and manual review.

### 4.2.1 Detection Chain

Type A uses a fixed five-stage sequence:



1. **Bucketization (60 seconds).** Trade records are aggregated to 60-second Time Buckets. Snapshot states are aligned with last-observation carry-forward.
2. **Feature construction (7 dimensions).** Activity uses `trade_count`, `amount_sum`, `avg_trade_size`; liquidity uses `spread`, `depth_drop`, `abs_depth_imbalance`; price uses `abs_midpoint_return`.
3. **Rolling normalization (window = 60 buckets).** Each feature is transformed with Rolling Z-Score against local history.
4. **Component fusion.**

$$S_{activity} = z_{trade\_count} + z_{amount\_sum} + z_{avg\_trade\_size}$$

$$S_{liquidity} = z_{spread} + z_{depth\_drop} + z_{abs\_depth\_imbalance}$$

$$S_{price} = z_{abs\_midpoint\_return}$$

5. **Anomaly thresholding (98th percentile).**

$$S_{stress} = S_{activity} + S_{liquidity} + S_{price}$$

A bucket is flagged when

$$S_{stress} \geq Q_{0.98}(S_{stress})$$

under the score distribution of the same market.

#### 4.2.2 Parameterization and Operational Choices

| Parameter          | Value        | Purpose                                                |
|--------------------|--------------|--------------------------------------------------------|
| Time Bucket width  | 60 s         | Preserves short bursts while reducing tick-level noise |
| Rolling window     | 60 buckets   | Provides a one-hour local baseline                     |
| Quantile threshold | 0.98         | Surfaces the top 2% stress buckets                     |
| Score fusion mode  | Additive sum | Keeps component contribution interpretable             |

The same parameter set is used across all six markets to keep comparisons stable.

#### 4.2.3 Empirical Output Summary

Batch execution on six markets produced an anomaly rate close to the 2% design target.

| Market ID | Trade Buckets | Anomalous Buckets | Anomaly Rate | Max Stress Score |
|-----------|---------------|-------------------|--------------|------------------|
|-----------|---------------|-------------------|--------------|------------------|

| Market ID | Trade Buckets | Anomalous Buckets | Anomaly Rate | Max Stress Score |
|-----------|---------------|-------------------|--------------|------------------|
| 1303357   | 243           | 5                 | 2.06%        | 13.20            |
| 1308327   | 1798          | 36                | 2.00%        | 23.57            |
| 1345814   | 741           | 15                | 2.02%        | 14.28            |
| 1345822   | 728           | 15                | 2.06%        | 19.74            |
| 572469    | 6145          | 123               | 2.00%        | 28.52            |
| 572473    | 2635          | 53                | 2.01%        | 35.78            |

Here, “Trade Buckets” refers to the bucket-level records retained for Type A scoring after preprocessing and feature construction, rather than the full aligned timeline reported in Table 5.1. The peak score is 35.78 in 572473, while the largest anomalous-bucket count is 123 in 572469. These anomalous buckets are bucket-level outputs of the Type A detector and can be consolidated into event-level stress windows for case-based analysis.

A representative event appears in 1308327 at 2026-02-13T20:00:00Z with composite stress 23.57, decomposed as Activity 2.62, Liquidity 14.65, and Price 6.30. The profile indicates a short-window stress interval dominated by order-book pressure.

### 4.3 Type B - Sniper Detection Clue Extraction

Type B analyzes wallet sessions. The target pattern is a large buy then sell round trip with short holding time and near break even outcome. Type A flags market stress at bucket level. Type B adds actor level clues with session identifiers and transaction hashes.

#### 4.3.1 Objective and Evidence Boundary

This detector is built for offline investigation. It ranks unusual sessions and keeps traceable evidence fields, including session\_id, trade direction, notional values, and anomaly score.

The method supports three bounded conclusions: rare sessions exist, a subset matches the sniper template, and the flagged sessions can be verified on chain. Identity attribution and intent attribution are outside scope.

#### 4.3.2 Detection Pipeline and Fixed Parameters

The module at submodules/algorithm/sniper\_detection uses a five stage pipeline.

1. Session splitting by trader\_address, with a new session after 30 minutes of inactivity.
2. Session level feature extraction: duration, trade count, volume, max trade, side order, and timing statistics.

3. Session embedding with all-MiniLM-L6-v2 to 384 dimensions.
4. Isolation Forest scoring with contamination 0.05 and 100 estimators.
5. Rule screening to keep large trade buy then sell sessions, then ranking by anomaly score.

| Parameter                           | Value            | Source                      |
|-------------------------------------|------------------|-----------------------------|
| Session gap threshold               | 30 minutes       | config.py, session_split.py |
| Large trade threshold               | 5000 USD         | config.py                   |
| Max holding time for canonical rule | 300 seconds      | config.py                   |
| Embedding model                     | all-MiniLM-L6-v2 | config.py                   |
| Embedding size                      | 384              | config.py                   |
| Isolation Forest contamination      | 0.05             | config.py, train_model.py   |
| Isolation Forest estimators         | 100              | config.py, train_model.py   |

#### 4.3.3 Output Scale and Candidate Definition

Current module outputs report 54,582 orders, 27,188 sessions, and 1,360 anomaly flagged sessions. The anomaly share is 5 percent by configuration.

The file results/strict\_sniper\_candidates.csv contains 40 rule screened candidates. This filter keeps sessions with large trades, BUY as first side, SELL as last side, and at least two trades.

A narrower validation layer is used for report ready sniper evidence. The canonical profile requires buy then sell direction, material notional size, short duration, near break even profit and loss, and a low tail anomaly score.

#### 4.3.4 Verified Cases Top 3 Canonical Sessions

The module case summary confirms three high confidence sessions, ranked 2 to 4. Rank 1 in the candidate file has 12 trades, so it is excluded from the two trade canonical examples.

| Rank | Session ID<br>truncated | Duration | Buy Amount   | Sell<br>Amount  | Net Profit<br>and Loss | Anomaly<br>Score |
|------|-------------------------|----------|--------------|-----------------|------------------------|------------------|
| 2    | 0x28d47763<br>...ba55_1 | 74 s     | 17894.13 USD | 17876.22<br>USD | -17.91<br>USD          | -0.4933          |
| 3    | 0x63b81ddc<br>...f906_1 | 76 s     | 17754.11 USD | 17736.34<br>USD | -17.77<br>USD          | -0.4870          |
| 4    | 0x6a4aaf27.<br>..f069_1 | 72 s     | 17787.85 USD | 17770.04<br>USD | -17.81<br>USD          | -0.4847          |

These cases cluster in holding time, notional size, and loss magnitude. Losses are about 0.10 percent of buy notional, consistent with a rapid round trip that does not target directional profit.

#### *4.3.5 Interpretation and Limits*

Type B provides behavioral clues, not legal or causal proof. The supported pattern is repeated: large buy, short hold, immediate sell, and near zero economic return.

This signal is useful for forensic triage and for cross checking Type A stress windows. A stronger coordination claim requires external wallet linkage evidence.

---

### 4.4 Visualization & Interactive Evidence View

The visualization layer transforms algorithmic outputs into navigable evidence packages. Raw anomaly scores, z-score vectors, and Isolation Forest outputs require structured presentation for human interpretation. The Streamlit-based frontend reads pre-computed artifacts from disk and renders them through two distinct visual patterns: a three-layer drill-down for Type A Market Stress events and a card-based layout for Type B Sniper candidates.

#### *4.4.1 Type A Visualization: Three-Layer Drill-Down*

Type A Market Stress events use a top-down hierarchical navigation pattern. Each layer reduces scope and increases informational density. Data sources for this view are the batch-processed outputs from the Algorithm module, organized under a per-market directory structure that contains market-level summaries, enriched event records, and time-bucket feature and score tables.

##### **Layer 1 -- Cross-Market Overview.**

The initial view presents two parallel panels. The left panel renders a ranked table of stress events sorted by peak stress score in descending order, showing market identifier, start timestamp, duration, and total traded amount. The right panel displays a horizontal bar chart comparing event counts per market, with a red gradient where darker bars indicate higher frequency. This level answers one question: which market has the most anomalous activity.

##### **Layer 2 -- Single-Market Event Inventory.**

After selecting a specific market, the view transitions to a per-market breakdown. Three components appear:

- **Event Attribution Pie Chart.** The dominant driver field partitions events into four categories: activity-driven, liquidity-driven, price-dislocation-driven, and mixed. This decomposition reveals whether a market's anomalies originate

from trade volume surges, order book degradation, price shocks, or concurrent multi-factor pressure.

- **Event Bubble Scatter Plot.** The x-axis encodes event start time. The y-axis encodes peak stress score. Bubble size maps to total traded amount. Bubble color indicates the dominant driver. Temporal clusters of large, same-colored bubbles suggest systematic anomalies rather than isolated incidents.
- **Event Detail Table.** A tabular list shows start/end timestamps, duration, peak stress score, and dominant driver for each event. Selecting a row triggers navigation to Layer 3.

### **Layer 3 -- Single-Event Micro-Diagnostics.**

The deepest view renders four vertically stacked subplots that share a common time axis. The displayed time window extends 10 minutes before and after the event boundaries to provide baseline context. A semi-transparent background rectangle highlights the event interval across all four subplots.

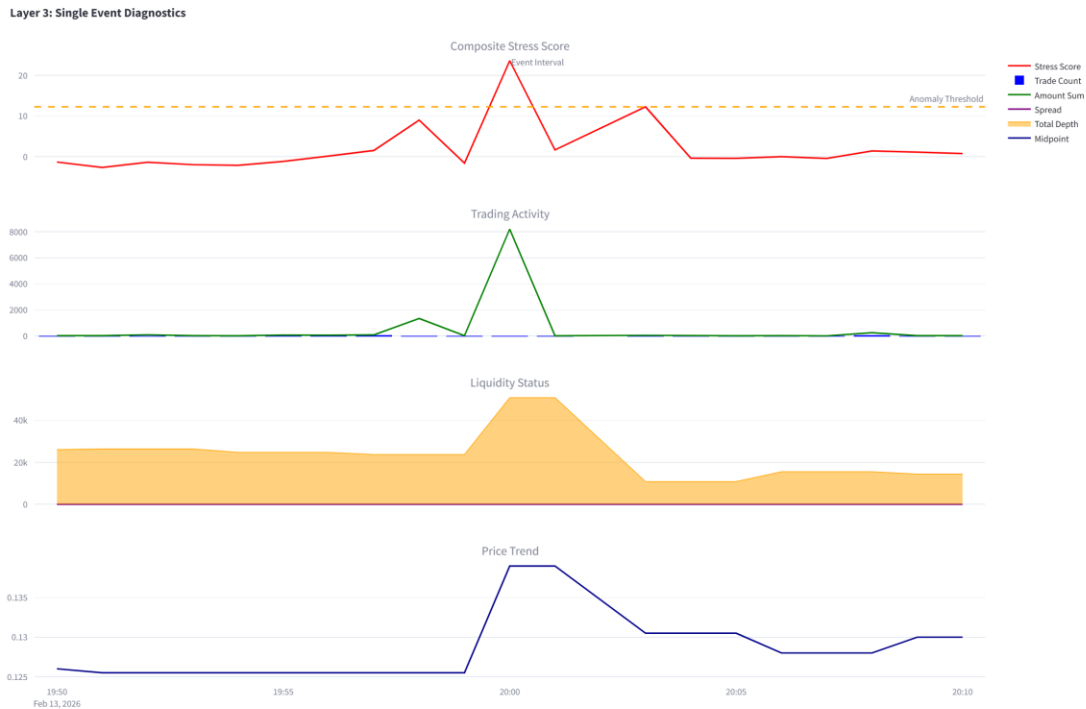
*Subplot 1 (Composite Stress Score)* draws the stress score trajectory as a line chart with a dashed horizontal line at the 98th-percentile anomaly threshold. The event interval shows where and how far the score exceeded normal bounds.

*Subplot 2 (Trading Activity)* combines a bar chart of per-bucket trade count with a secondary-axis line for total notional amount. The bar height reveals transaction frequency; the line position reveals capital intensity. Divergence between the two (many small trades vs. few large trades) distinguishes retail-driven surges from concentrated participant behavior.

*Subplot 3 (Liquidity Status)* pairs a line for bid-ask spread with a filled-area chart for total order book depth on a secondary axis. Rising spreads and collapsing depth during the event window indicate that market makers withdrew quotes or that informed traders consumed liquidity faster than replenishment.

*Subplot 4 (Price Trend)* plots the bid-ask midpoint as a single line. A flat midpoint during periods of elevated activity and liquidity stress suggests failed price manipulation (activity without successful impact). A concurrent midpoint shift strengthens the evidence that the event materially affected market consensus.

This four-panel layout supports causal attribution within a single glance. If Subplots 2 and 3 show disturbances while Subplot 4 remains flat, the anomaly may represent a volume spike without price consequence. If all four panels deviate simultaneously, the event carries higher evidentiary weight.



#### 4.4.2 Type B Visualization: Card-Based Evidence View

Type B Sniper candidates use a card-based "Detective Report" layout. Data sources are the sniper detection output directory containing a strict candidate table (40 records), a verified case detail file (3 entries), a full candidate collection (40 entries), and an image subdirectory.

**Summary Metrics Row.** The view opens with three aggregate indicators: total candidate count (40), mean anomaly score across candidates, and verified case count (3). Below this row, a histogram of anomaly scores shows the distribution shape of the scored session population.

**Verified Case Cards.** Three dedicated cards display the verified sniper cases (ranked 2, 3, and 4). Each card shows:

- Rank position and session identifier
- Session duration (74s, 76s, 72s)
- Anomaly score
- Buy/sell amounts and net P&L

An expander toggle reveals additional detail: execution prices, timestamps, transaction hashes with Polygonscan URLs, and the attack window visualization image. The three verified case images display the spike-and-decay timeline shape described in Section 4.3.5.

**Candidate List with Pagination.** Below the verified cases, a paginated list presents all 40 strict candidates at five cards per page. Pagination state is managed through Streamlit's session state. Each card displays session ID, trade count, total volume, anomaly score, and a suspicious rank. Risk-tier color coding uses three levels: red for anomaly scores below -0.6, orange for scores between -0.6 and -0.5, and green for scores at or above -0.5. This color scheme allows rapid visual triage of high-risk candidates without reading individual score values.

4.4.3 Data Contract with Algorithm Module

The visualization layer does not execute algorithms. It reads standardized output artifacts produced by the Algorithm module. The interface contract specifies data artifact locations, required columns, and format expectations.

**Type A Contract.** The Algorithm module writes batch results to a per-market output directory. Five artifact categories are required per market:

| Artifact               | Key Fields                                                                                         | Visualization Layer           |
|------------------------|----------------------------------------------------------------------------------------------------|-------------------------------|
| Market stress summary  | market identifier, event count, max stress score, anomaly rate                                     | Layer 1 cross-market overview |
| All-events aggregate   | enriched event columns across all markets                                                          | Layer 1 top-event ranking     |
| Enriched event records | start/end timestamps, duration, peak stress score, total amount, dominant driver, component scores | Layer 2 event inventory       |
| Time-bucket features   | bucket index, datetime, trade count, amount, spread, total depth, midpoint                         | Layer 3 micro-diagnostics     |
| Time-bucket scores     | bucket index, datetime, stress score, anomaly flag, component z-scores                             | Layer 3 threshold overlay     |

**Type B Contract.** Sniper detection data resides in a dedicated subdirectory within the visualize module. Four artifact categories are required:

| Artifact                  | Records    | Visualization Layer        |
|---------------------------|------------|----------------------------|
| Strict candidate table    | 40 rows    | Paginated candidate list   |
| Verified case details     | 3 entries  | Verified case detail cards |
| Full candidate collection | 40 entries | Sorting and filtering      |
| Attack window images      | 3          | Case visualization         |

Error handling follows a graceful degradation pattern. If a required artifact is missing, the view logs a warning, returns an empty DataFrame, and displays a user-facing message indicating which data is unavailable rather than failing with an unhandled exception.

4.4.4 Performance Optimization Strategy

The frontend targets sub-three-second render time for any user interaction (market switch, view change, card expansion, pagination advance). This target is met through four mechanisms.

**File-Level Caching.** All data loading functions carry the `@st.cache_data` decorator. Streamlit computes a hash of function arguments and file modification timestamps. Disk reads occur only on first access or when the underlying file changes.

**Frontend Pagination.** Sniper candidate cards render five at a time rather than all 40 simultaneously. This reduces DOM node count and prevents browser slowdown on large candidate sets.

**Conditional Rendering.** View modules check data availability before constructing Plotly figures. Empty datasets trigger a lightweight "No data available" message instead of attempting to render empty charts.

**Lazy Detail Loading.** Verified case details (transaction hashes, attack window images) load only when the user expands the card expander. This defers I/O for data that most users will not inspect.

---

5. Validation, Testing & Empirical Analysis

5.1 Dataset & Empirical Approach

The empirical study uses historical Polymarket data from six markets: 1303357, 1308327, 1345814, 1345822, 572469, and 572473. The set spans cryptocurrency, political, economic, and geopolitical contracts. Trade logs and order-book snapshots are aligned to 60-second Time Buckets before analysis.

Table 5.1: Market Dataset Summary

| Market ID | Market Name                | Domain         | Trade Records | Time Buckets |
|-----------|----------------------------|----------------|---------------|--------------|
| 1303357   | ETH Price Range            | Cryptocurrency | 3703          | 3343         |
| 1308327   | Government Shutdown        | Geopolitical   | 10627         | 724          |
| 1345814   | Fed Rate Cut Path          | Economic       | 3205          | 22           |
| 1345822   | CPI Release Outcome        | Economic       | 3208          | 23           |
| 572469    | Trump Fed Chair Nomination | Political      | 14705         | 3348         |
| 572473    | BTC Price Range            | Cryptocurrency | 19134         | 3344         |



Type A flags bucket-level Market Stress Anomalies with a 98th-percentile threshold. Type B scores wallet sessions and keeps only strict buy-then-sell candidates with short holding periods and large notional size. The validation goal is narrow: test whether flagged outputs remain statistically extreme and whether the top cases remain interpretable under manual review.

Two corpus-level figures anchor the results. Market 1308327 reaches a peak stress score of 23.57 near its resolution window. Market 572473 reaches the corpus maximum of 35.78, which is higher than the peak observed in 572469.

## 5.2 Case Study Validation

### 5.2.1 Integrated Case: Trump Fed Chair Market

Market 572469 is the main validation case because it contains the densest overlap between the two detection layers. Type A first flags 123 anomalous buckets in this market, which are then consolidated into 115 stress events for event-level review. Its peak Composite Stress Score is 28.52. Type B identifies a confirmed sniper session with a 74-second holding period and an Isolation Forest score of -0.4933.

Type A evidence is concentrated in a short cluster. The peak event occurs on 2026-02-16 20:49 UTC, spans 120 seconds, records 15 trades totaling \$680.81, and is classified as liquidity-driven. Among the 115 events, 97 have supporting snapshot coverage, which allows activity, liquidity, and price contributions to be separated.

Type B provides a concrete wallet-level clue.

| Attribute            | Value                   |
|----------------------|-------------------------|
| Wallet (truncated)   | 0x28d47763...ba55       |
| Session structure    | 2 trades, BUY then SELL |
| Buy amount           | \$17,894.13             |
| Sell amount          | \$17,876.22             |
| Holding period       | 74 seconds              |
| Net P&L              | -\$17.91                |
| Total session volume | \$35,770.35             |
| Anomaly score        | -0.4933                 |
| Sniper label         | is_sniper = 1           |

The joint signal is useful, but it does not prove causation. The overlap between the sniper session and the stress window can be explained in at least three ways: the wallet activity contributed to the stress score, both signals reacted to the same external event, or the overlap occurred inside an already active market. The report can support the overlap claim. It cannot support a stronger causal claim with the current dataset.

### 5.2.2 Validation Summary Table

| Dimension        | Raw CSV Data               | Isolated Numerical Scores | This System's Output                    |
|------------------|----------------------------|---------------------------|-----------------------------------------|
| Readability      | Low                        | Medium                    | High                                    |
| Interpretability | Manual inspection required | Limited                   | Component-level explanation per event   |
| Context          | None                       | Narrow                    | Timeline and cross-layer linkage        |
| Interactivity    | None                       | None                      | Drill-down review through the interface |

The empirical result is simple. The system preserves the strongest anomalies, keeps them inspectable, and links market-level symptoms to wallet-level clues without claiming more than the data can support.

---

## 6. Real-World Value & Conclusion

### 6.1 Practical Value

The system is an offline investigation tool. Its value comes from four direct uses.

| Use Case                | Practical Value                                                                            |
|-------------------------|--------------------------------------------------------------------------------------------|
| Market investigation    | Surfaces abnormal windows and suspicious wallet sessions for manual review                 |
| Research support        | Provides a fixed six-market baseline for prediction-market anomaly analysis                |
| Explainable auditing    | Preserves component scores and rule-based filters so each flag can be traced               |
| Cross-market comparison | Keeps parameters fixed across markets, which makes event counts and peak scores comparable |

The tool is useful because it converts static records into evidence that can be reviewed without rebuilding the pipeline. That is the main contribution. It is not an execution system, an alerting platform, or a causal inference engine.

---

### 6.2 Conclusion

This project builds a two-layer anomaly analysis workflow for historical Polymarket data. Type A detects Market Stress Anomalies from bucket-level trading and liquidity signals. Type B detects Sniper Attack patterns from wallet sessions. Across

six markets, the strongest integrated evidence appears in market 572469, which combines 123 anomalous buckets consolidated into 115 stress events, a peak stress score of 28.52, and a confirmed 74-second sniper session with anomaly score -0.4933. The broader corpus peak is 35.78 in market 572473.

The current system has clear limits. It is offline only. Its output depends on data completeness. Parameter choices were calibrated manually rather than optimized against labeled ground truth. Cross-layer overlap can be reported, but causation cannot be established from the current evidence alone. Transferability beyond Polymarket also remains untested.

The result is still useful. The pipeline converts raw market records into reviewable anomaly evidence, keeps the strongest cases visible, and gives investigators a compact path from market-wide symptoms to wallet-level clues.

## 7. Appendix: AI Collaboration Statement

### 7.1 AI Toolchain and Responsibility Split

The project used a two-layer AI workflow with fixed decision boundaries.

| Layer                     | Tooling                                              | Primary Function                                                            | Acceptance Owner |
|---------------------------|------------------------------------------------------|-----------------------------------------------------------------------------|------------------|
| Architecture and planning | OpenCode + DeepSeek API                              | Requirement exploration, scope framing, design reasoning, and plan drafting | Human team       |
| Execution                 | TRAE (SOLO mode)                                     | Task execution, command runs, file updates, and validation runs             | Human team       |
| Skill orchestration       | Superpowers skills plus planning-with-files workflow | Structured planning, subtask dispatch, and persistent progress tracking     | Human team       |

The boundary was explicit. AI generated proposals and intermediate outputs. Human reviewers accepted or rejected each integration step.

### 7.2 Skill-Based Workflow Applied

The collaboration process followed the sequence defined in the project AI SOP.

| Stage   | Skill or Method           | Output                                    | Persistent Record |
|---------|---------------------------|-------------------------------------------|-------------------|
| Stage A | superpowers/brainstorming | Goal, Non-Goal, constraints, initial risk | findings.md       |

| Stage   | Skill or Method                         | Output                                                                 | Persistent Record                      |
|---------|-----------------------------------------|------------------------------------------------------------------------|----------------------------------------|
|         |                                         | notes                                                                  |                                        |
| Stage B | superpowers/planning                    | Granular development tasks with verification checkpoints               | task_plan.md                           |
| Stage C | superpowers/subagent-driven-development | Parallelized subtask execution with evidence per subtask               | progress.md plus task status updates   |
| Stage D | planning-with-files                     | Continuous synchronization of decisions, task states, and failure logs | findings.md, task_plan.md, progress.md |

A subtask was closed only when three conditions were met: implementation finished, validation evidence recorded, and task state updated.

### 7.3 Constraint Control with AGENTS.md

The development module stores execution rules in documents/AGENTS.md. This file covers environment setup, code conventions, data pipeline boundaries, testing checkpoints, and module-level parameter notes.

The constraint set targets three failure modes: inconsistent code style across submodules, real-time API calls leaking into the static visualization layer, and undocumented threshold changes. Each rule maps to a concrete enforcement point.

| Constraint Domain     | Rule                                                                        | Enforcement Point                          |
|-----------------------|-----------------------------------------------------------------------------|--------------------------------------------|
| Architecture boundary | No real-time API calls in visualization                                     | Code review before commit                  |
| Code style            | Import grouping, snake_case naming, config externalization                  | Consistent pattern check across submodules |
| Data contract         | Algorithm outputs serializable to JSON; graceful handling of missing data   | Frontend load validation                   |
| Parameter management  | All thresholds defined in config.py; no hardcoded values                    | Config file audit                          |
| Testing gate          | JSON artifacts loadable by Streamlit; visualization renders before delivery | Pre-delivery check                         |

## 7.4 Human Oversight and Decision Rights

AI tools were productivity aids. Final authority remained human.

All accepted outputs were reviewed by team members before integration. Suggestions conflicting with the project threat model or detection criteria were discarded. Empirical claims in this report are tied to computed outputs and documented case evidence.

## 7.5 Correction Case and Outcome

A draft AI proposal reduced sniper detection to a simple two-condition rule.

That proposal was rejected because it did not satisfy the project definition of session structure, direction sequence, profit behavior, and anomaly scoring. The accepted revision restored the defined pipeline and retained the verified case with 74-second duration, 17,894.13 USD buy amount, 17,876.22 USD sell amount, -17.91 USD net P and L, and -0.4933 anomaly score.

This correction illustrates the operating boundary. AI accelerated drafting and execution, while model choice, detection logic, and claim acceptance remained under human control.

---

## References

- [1] obra, "superpowers." GitHub. Accessed: Apr. 18, 2026. [Online]. Available: <https://github.com/obra/superpowers>
- [2] "agents.md." Accessed: Apr. 18, 2026. [Online]. Available: agents.md
- [3] OthmanAdi, "planning-with-files." GitHub. Accessed: Apr. 18, 2026. [Online]. Available: <https://github.com/OthmanAdi/planning-with-files>
- [4] Streamlit Inc., "Streamlit Documentation." Accessed: Apr. 18, 2026. [Online]. Available: <https://docs.streamlit.io/>
- [5] OpenCode.ai, "OpenCode.ai Documentation." Accessed: Apr. 18, 2026. [Online]. Available: <https://opencode.ai/docs/en>